

Prompt Evolution GA

Deep Dive — how the code works, end to end

Explainer generated for Salman Adnan's portfolio

This document explains `prompt-evolution-ga` at a level meant for a complete beginner: every technical term is defined the first time it is used, every module is walked through, and every diagram shows a real flow in the actual code. Nothing here is invented — every claim traces back to a specific file, and anywhere the code or data is ambiguous, it is flagged explicitly rather than guessed at.

Contents

1	What this project is	3
1.1	The central idea: what is being evolved?	3
1.2	Why this fitness function matters	3
2	System architecture	4
2.1	The project's files	4
2.2	How the pieces connect	4
3	Configuration — <code>config.py</code>	6
4	The dataset — <code>dataset.py</code> and <code>Dataset/APPS/</code>	7
4.1	<code>load_problems()</code>	7
4.2	<code>split()</code> — stratified train/holdout split	7
5	The GA core — <code>ga.py</code>	8
5.1	The Individual	8
5.2	One generation, visually	8
5.3	Selection schemes	9
5.4	The breeding pipeline — <code>breed()</code>	9
5.5	Fitness computation — <code>compute_fitness()</code>	10
5.6	Checkpointing and resume	10
5.7	Early stopping (convergence)	10
6	Talking to the LLMs — <code>llm.py</code>	12
6.1	Authentication	12
6.2	The two call sites	12
6.3	Retries and backoff	12
6.4	Reasoning/thinking tokens	12
7	Scoring generated code — <code>evaluator.py</code>	12
7.1	Extracting code from the model's answer	13
7.2	Compiling	13
7.3	Running against test cases	13
7.4	Early stopping per problem	13
7.5	Score reasons	13
8	Cost accounting — <code>cost_tracker.py</code>	14
9	Entry point and orchestration — <code>main.py</code>	14

10 Analysis scripts	14
10.1 <code>run_baseline_eval.py</code>	15
10.2 <code>visualize_test_results.py</code>	15
11 Results, as actually recorded in this repository	16
11.1 Headline numbers	16
11.2 Cost	16
11.3 The five comparison figures	16
12 Known limitations (from the project's own documentation)	16
13 Glossary	17

1 What this project is

In one sentence: this project uses a *genetic algorithm* to automatically search for the best possible wording of an instruction (“system prompt”) that tells an AI model how to write C++ solutions to competitive-programming problems, judging each candidate wording by actually compiling and running the code it produces.

Term: Genetic Algorithm (GA)

A search technique inspired by biological evolution. You start with a *population* of candidate solutions, score each one with a *fitness function*, and repeatedly breed a new generation by combining (*crossover*) and randomly tweaking (*mutation*) the best-scoring candidates. Over many *generations* the population tends to improve, because better-scoring candidates are more likely to be selected as parents.

Term: Large Language Model (LLM)

A machine learning model trained on huge amounts of text that can generate new text — including computer code — in response to a prompt. This project calls two LLMs from Google’s Gemini family, both hosted on Google Cloud’s *Vertex AI* platform (Google’s managed service for running LLMs in the cloud):

- **gemini-2.5-flash** — the **target model**, the one being tested. It writes the actual C++ code, guided by whatever prompt is currently being evaluated.
- **gemini-2.5-pro** — the **optimizer model**, a stronger and more expensive model used only to perform crossover and mutation *on the text of the prompts themselves* (see Section 5).

1.1 The central idea: what is being evolved?

In a textbook GA, a *chromosome* is usually a fixed-length string of numbers or bits. Here, the chromosome is something unusual: **an entire paragraph of English instructions** — the system prompt that will be prepended to every code-generation request. For example, one of the five hand-written starting prompts (in `seeds.py`) reads:

“You are an expert competitive programmer. Before writing any code, analyse the problem carefully: identify the input format, the output format, the constraints, and any edge cases (empty input, n=1, maximum values, overflow risk)...”

The GA’s job is to discover wording that makes **gemini-2.5-flash** produce more correct C++ programs, without a human manually trying different phrasings by hand.

Term: Fitness function

A function that assigns a single number to a candidate solution, measuring how good it is. Here, the fitness of a prompt is the **mean pass rate**: for a fixed pool of competitive-programming problems, generate C++ using that prompt, compile it, run it against each problem’s real test cases, and average the fraction of test cases passed across all problems. A fitness of 1.0 means every test case of every problem passed.

1.2 Why this fitness function matters

The project deliberately avoids asking another AI model “does this code look correct?” — that would just move subjective judgement around instead of removing it. Instead, correctness is decided the same

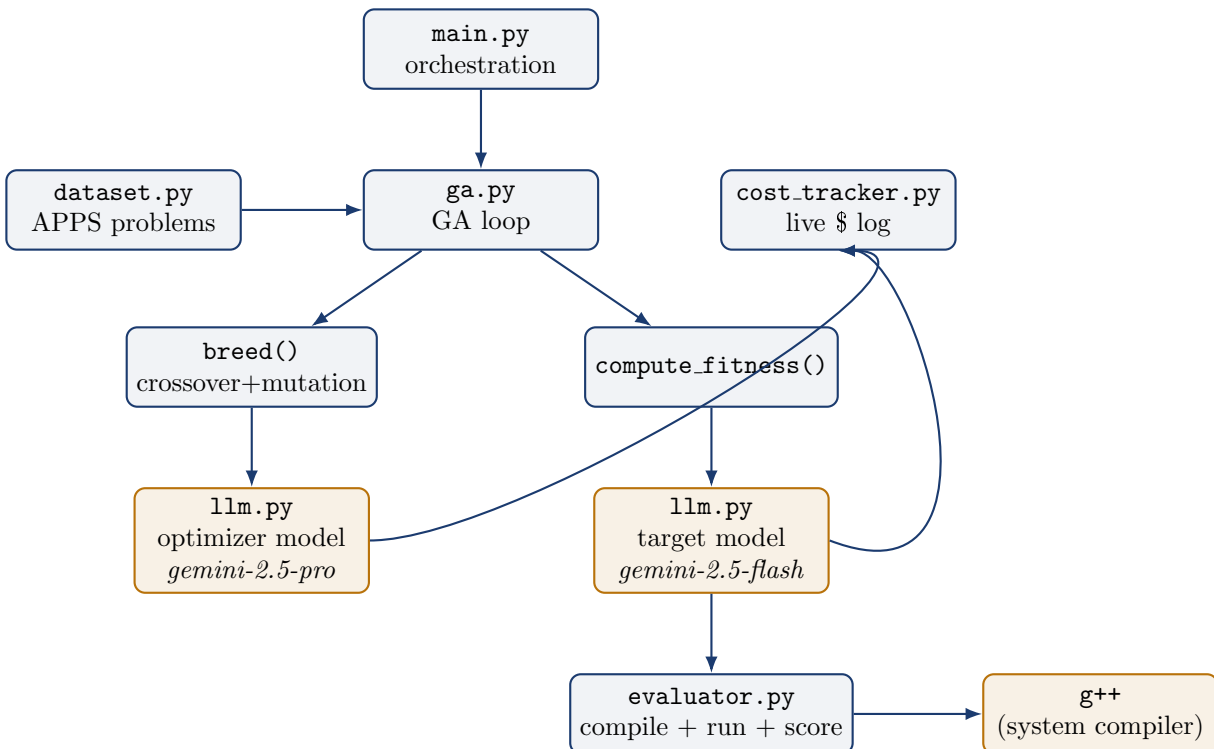
way a real judge (e.g. Codeforces) would: by compiling with `g++` and running the binary against real test cases. This is implemented in `evaluator.py` (Section 7).

2 System architecture

2.1 The project's files

<code>prompt-evolution-ga/</code>	
<code>config.py</code>	<i>all hyperparameters & model names</i>
<code>seeds.py</code>	<i>15 hand-written starting prompts</i>
<code>dataset.py</code>	<i>loads & splits APPS problems</i>
<code>llm.py</code>	<i>calls to Gemini via Vertex AI</i>
<code>evaluator.py</code>	<i>compile, run, score C++ code</i>
<code>ga.py</code>	<i>the GA loop itself</i>
<code>cost_tracker.py</code>	<i>per-call \$ accounting</i>
<code>main.py</code>	<i>entry point / orchestration</i>
<code>run_baseline_eval.py</code>	<i>evolved vs. baseline test</i>
<code>visualize_test_results.py</code>	<i>builds comparison figures</i>
<code>Dataset/APPS/</code>	<i>problem statements + test cases</i>
<code>results/</code>	<i>3 completed runs + comparison</i>
<code>Costs/</code>	<i>real \$ reports from those runs</i>

2.2 How the pieces connect



How to read this diagram

`main.py` starts everything and owns the top-level loop over generations. It hands the population to `ga.py`, which does two jobs per generation: breed new individuals (talking to the *optimizer* model) and score them (talking to the *target* model, then compiling/running with `g++` through `evaluator.py`). Every LLM call, on either path, is logged through `cost_tracker.py` so that the running dollar cost is always known.

3 Configuration — config.py

Every tunable constant in the project lives in this one file. This is a common pattern called **centralized configuration**: instead of scattering magic numbers through the code, they are declared once and imported everywhere (`import config; config.POPULATION_SIZE`).

Constant	Value	What it controls
EVOLUTION_SIZE	65	problems used to score prompts during evolution
HOLDOUT_SIZE	200	problems reserved for the final, unbiased test
RANDOM_SEED	42	seeds every random-number generator, so the train/test split and breeding choices are reproducible
POPULATION_SIZE	5	number of prompts alive in each generation
GENERATIONS	12	maximum number of breeding rounds
ELITISM_COUNT	0 (default)	number of top individuals copied unchanged into the next generation (see §5.3)
CROSSOVER_ONLY_SLOTS	2	first 2 offspring per generation get crossover only, no mutation
TOURNAMENT_SIZE	3	contestants sampled per tournament-selection draw
CROSSOVER_RATE	0.8	probability that two chosen parents are actually merged (vs. one being copied)
MUT_INJECT_PROB	0.25	probability of the “add a new strategy” mutation
MUT_DELETE_PROB	0.15	probability of the “remove one sentence” mutation
MUT_REPHRASE_PROB	0.10	probability of the “reword, keep meaning” mutation
TARGET_MODEL	gemini-2.5-flash	writes the C++ code being judged
OPTIMIZER_MODEL	gemini-2.5-pro	performs crossover/mutation on prompt text
TARGET_TEMPERATURE	0.0	target model is deterministic (no randomness in code generation)
OPTIMIZER_TEMPERATURE	0.7	optimizer model is allowed some creative variation
API_FALLBACK_MAX_TOKENS_CODE	8192	max length of generated C++
API_FALLBACK_MAX_TOKENS_OPS	700	max length of a crossover/mutation result (flagged as a real bug, §12)
CONVERGENCE_CV_THRESHOLD	0.01	population judged “homogeneous” once its coefficient of variation drops below this
CONVERGENCE_PATIENCE	3	generations of homogeneity in a row before early stopping
EXEC_TIMEOUT	2 s	per-test-case time limit
COMPILE_TIMEOUT	15 s	compile time limit
PARALLEL_EVALS	5	worker threads used for concurrent fitness evaluation

Term: Token

LLMs do not read/write in whole words; they break text into sub-word chunks called tokens. API cost and length limits (like `API_FALLBACK_MAX_TOKENS_CODE`) are measured in tokens, roughly $\frac{3}{4}$ of a word each in English.

Term: Coefficient of variation (CV)

A measure of how spread out a set of numbers is, computed as the standard deviation divided by the mean ($CV = \sigma/\mu$). A CV near 0 means all values are nearly identical (the population has become homogeneous); a high CV means the population is still diverse. `ga.py` uses this to decide when to stop early (§5).

4 The dataset — `dataset.py` and `Dataset/APPS/`

Term: APPS benchmark

APPS (Hendrycks et al., 2021) is a published academic dataset of programming problems with reference test cases, used to measure how well AI models write code. This project includes a small hand-picked subset of it under `Dataset/APPS/`, cited in `Dataset/APPS/README.txt`. It remains under the original dataset’s license terms (separate from this project’s own MIT license).

Each problem is a folder containing `question.txt` (the natural-language problem statement) and `input_output.json` (a list of test-case inputs and their expected outputs). `dataset.py` defines, by hand, exactly which problem folders to use (dictionary `_SELECTED`), organized by **difficulty rating** — a numeric Codeforces-style score (800 = easy, up to 1900 = hard in this subset).

4.1 `load_problems()`

Reads every selected folder’s `question.txt` and `input_output.json`, skipping (with a warning) any folder that is missing files, has invalid JSON, or has empty test cases, and returns a flat list of problem dictionaries: `{id, question, inputs, outputs, rating}`.

4.2 `split()` — stratified train/holdout split

Term: Stratified sampling

Splitting a dataset so that each subgroup (here, each difficulty rating) is represented proportionally in every split, rather than splitting purely at random (which could accidentally put all the hard problems in one split).

`split()` builds the 65-problem *evolution pool* by: (1) always including every “harder” training problem (rating ≥ 1800), and (2) for each remaining rating tier, shuffling (with the fixed seed 42) and taking a fixed count per tier from `_EVOLUTION_PER_TIER`. Everything left over is shuffled and the first 200 become the *holdout* (test) set — the problems the GA never sees during evolution, used only once at the very end to measure the winning prompt honestly.

Minor discrepancy, low stakes

`main.py` prints “loading 305 pre-selected Codeforces problems”, but the dataset actually committed in this copy of the project contains 300 problem folders. This is called out in the README’s own “Minor hygiene” note — it is a stale print string, not a functional bug (the split logic works off however many problems actually load).

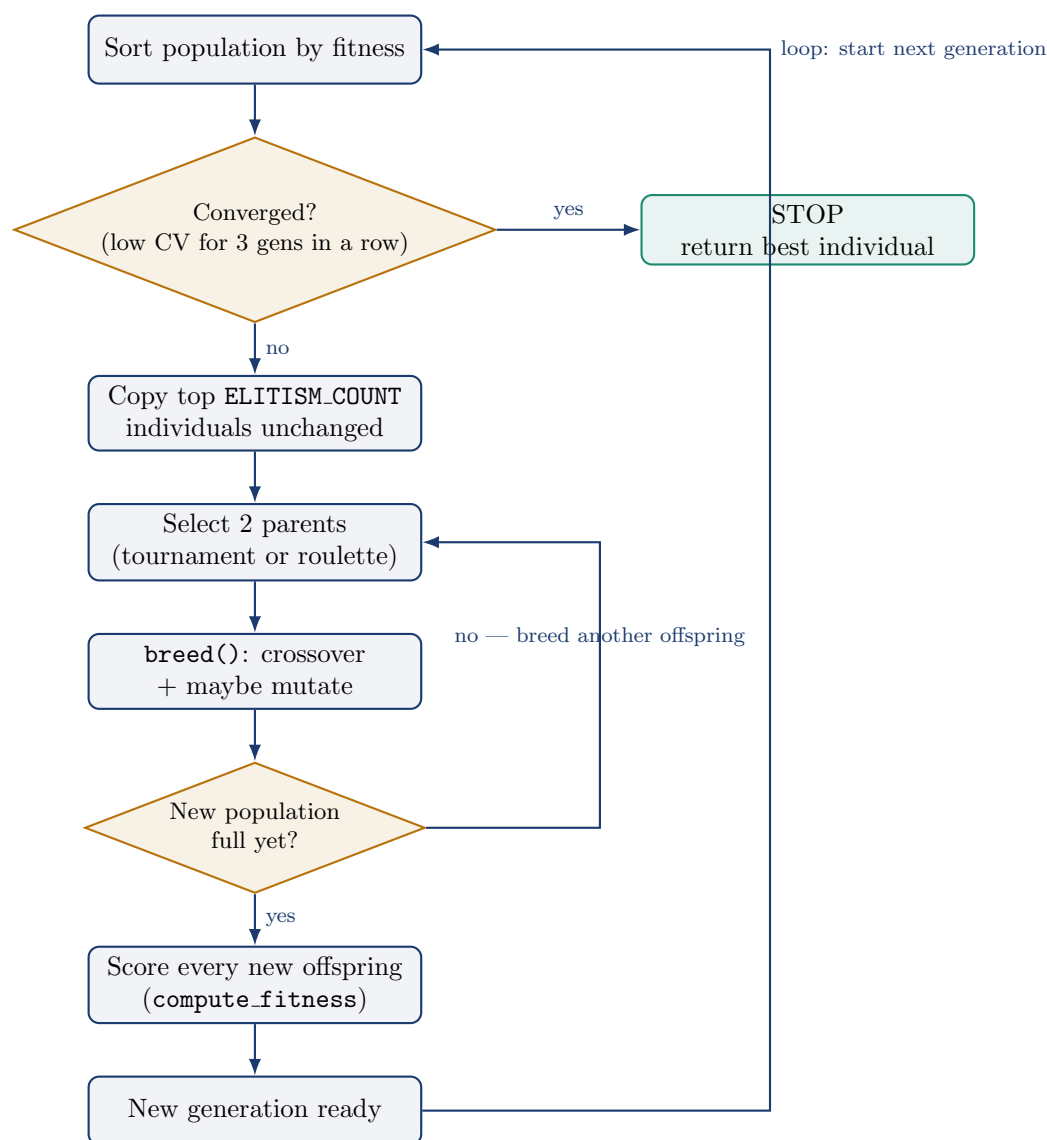
5 The GA core — `ga.py`

5.1 The Individual

Term: Individual / chromosome

One candidate solution in the population. Here it is a Python `dataclass` (a class whose main job is holding data) with the fields: `prompt` (the instruction text), `fitness`, a unique `uid`, its two parents' `uids`, which `operators` produced it (e.g. "`crossover+inject`"), the `intermediate_steps` (the prompt text after each operator, for debugging/genealogy), and `failure_examples` (up to 5 real failing test cases from its own evaluation, used to guide future mutation — see §5.4).

5.2 One generation, visually



5.3 Selection schemes

Term: Selection

The stepbox that decides which individuals get to be parents of the next generation. Fitter individuals should be more likely to be picked, but not *certain* to be picked — some randomness keeps the population diverse.

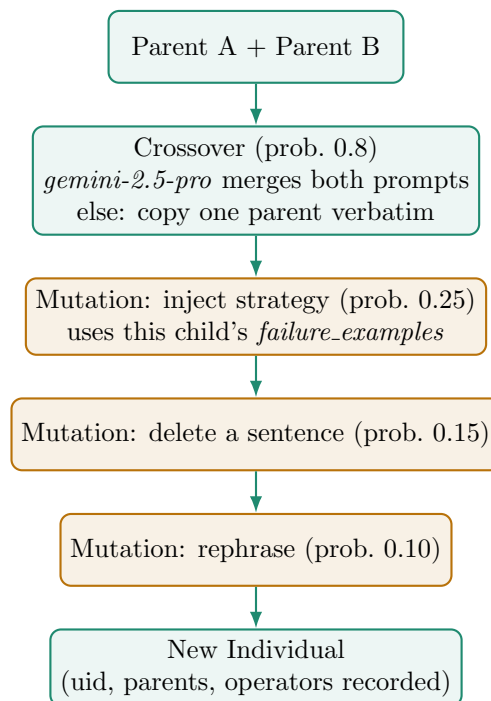
Tournament selection (`tournament_select`, default scheme): pick `TOURNAMENT_SIZE` (3) individuals uniformly at random from the population, and return whichever of those 3 has the highest fitness.

```
def tournament_select(population, rng):
    contestants = rng.sample(population, config.TOURNAMENT_SIZE)
    return max(contestants, key=lambda x: x.fitness)
```

Roulette-wheel selection (`roulette_select`): every individual gets a slice of a "wheel" proportional to its fitness (fitness is clamped to ≥ 0 first, so a failed individual with negative fitness cannot get a negative-size slice); a single random spin decides the parent. If every fitness is exactly 0, a uniformly random individual is picked instead of dividing by zero.

Elitism (activated automatically when `--scheme elitism` is passed, which sets `elitism_count=1`): the single best individual from the current generation is copied into the next generation *untouched*, guaranteeing fitness never regresses, before tournament selection fills the remaining slots. This is why the project has three selection schemes to compare (tournament, roulette, and tournament-with-elitism) rather than two.

5.4 The breeding pipeline — `breed()`



Each mutation stepbox is an independent coin flip (mutations can stack: 0, 1, 2, or all 3 can fire on the same child), except for the first `CROSSOVER_ONLY_SLOTS` (2) offspring of every generation, which skip mutation entirely by construction (`allow_mutation=False` in `ga.py`'s `run()`), guaranteeing at least

two “pure crossover” children exist to compare against mutated ones. If any LLM call in the pipeline returns an empty string, its result is discarded and the prompt from before that step is kept (`res if res.strip() else prompt`), so a failed API call cannot silently blank out a prompt.

Term: Crossover

Combining traits from two parents into one offspring. In a classic GA on bit-strings this literally means splicing two bit sequences together. Here it means asking the optimizer LLM to read both parent prompts and write one unified prompt that “preserves the strongest strategies from each” (see the literal prompt template in `llm.py`’s `_CROSSOVER_USER`).

Term: Mutation

A small, mostly-random change applied to an offspring, meant to introduce variation that selection alone would not produce. Three kinds exist here, each its own LLM call: *inject* (add one new strategy sentence, optionally steered by real recent failures), *delete* (remove the least useful sentence), and *rephrase* (reword without changing meaning).

5.5 Fitness computation — `compute_fitness()`

For a given individual and the 65-problem evolution pool, this function evaluates all 65 problems **concurrently** using a `ThreadPoolExecutor` with `PARALLEL_EVALS` (5) worker threads, each running `_eval_one_problem` (call the target LLM, then `evaluator.run_tests_verbose`). Fitness is the mean score across all 65 problems. It also calls `_pick_failures()`, which keeps up to 5 representative failures (prioritized: compile failures first, then total failures, then partial passes), stored on the individual for the *inject* mutation to use later.

Term: Thread pool / concurrency

Running multiple independent tasks (here, 65 separate LLM calls + compiles + test runs) at the same time using several worker threads, instead of one at a time, so the program is not sitting idle waiting on network responses. `ThreadPoolExecutor` is Python’s standard-library tool for this.

5.6 Checkpointing and resume

After every generation, `main.py`’s `save_checkpoint()` writes a `generation_NN.json` file containing the full population (every field of every `Individual`) and the Python random-number generator’s internal state (`rng_state_to_dict`). `--resume` reloads the latest checkpoint, restores the population and RNG state exactly, and continues the loop from `last_gen + 1`. Because the RNG state is restored bit-for-bit, a resumed run makes the exact same random choices it would have made had it never stopped — this is what makes a multi-hour run safe to interrupt.

Generation 0 (the seed prompts) is checkpointed *per-seed* (`seed_NN-<uid>.json`) rather than all at once, so if the process is killed partway through evaluating the 5 starting seeds, already-evaluated seeds are not re-evaluated (and re-billed) on resume.

5.7 Early stopping (convergence)

After each generation’s stats are computed, if the population’s coefficient of variation (fitness spread) stays below `CONVERGENCE_CV_THRESHOLD` (0.01) for `CONVERGENCE_PATIENCE` (3) generations in a row, the loop breaks early on the theory that a population this uniform is unlikely to keep improving. All

three completed runs in **results/** ran the full 12 generations, so this trigger was not observed to have fired in the committed data (not independently confirmed from the progress logs, which are not part of what is committed).

6 Talking to the LLMs — llm.py

6.1 Authentication

The client authenticates to Vertex AI using a **GCP service-account key** — a JSON credentials file (expected at `./service-account.json`, never committed to the repo) identifying a Google Cloud service account with the “Vertex AI User” role.

Term: Service account

A non-human Google Cloud identity used by programs (rather than a person logging in) to call Google APIs, authorized by a downloadable private-key JSON file.

The client (`google.genai.Client`) is created lazily on first use and cached in a module-level variable guarded by a lock, so concurrent threads share one client instead of each creating their own.

6.2 The two call sites

`generate_code()` calls the target model with a strict `_CODE_GEN_DIRECTIVE` appended to the system prompt, forbidding markdown code fences or any explanatory text — the model must output raw C++ starting with its very first character. `crossover()`, `mutate_inject()`, `mutate_delete()`, and `mutate_rephrase()` all call the optimizer model with an analogous `_OPERATOR_DIRECTIVE` demanding plain prompt text with no labels or commentary.

6.3 Retries and backoff

`_call_api()` retries up to 10 times on: rate limiting (HTTP 429 / `RESOURCE_EXHAUSTED`), server errors (500/503/`UNAVAILABLE`), timeouts, and empty responses, each time waiting an **exponentially increasing** delay ($5 \times 2^{\text{attempt}}$ seconds plus a small random jitter) before trying again. Any other kind of exception is logged and also retried (up to the same limit) rather than crashing the whole run. If all 10 attempts fail, `generate_code()` raises `LLMCallError`, which the caller treats as an `infra_failure` (scored as if every test failed, but tagged distinctly so it is not confused with the model actually writing wrong code).

Term: Exponential backoff

A retry strategy where each failed attempt waits longer than the last (typically doubling), so that repeated failures — often caused by the server being temporarily overloaded — do not make things worse by hammering it faster and faster.

6.4 Reasoning/thinking tokens

Both models are called with a `ThinkingConfig` (`thinking_budget`: 512 for code generation, 2048 for prompt operators). This is Gemini’s internal “chain-of-thought” reasoning step before producing its final answer; those internal tokens are billed at the output token rate and are added into the recorded `output_tokens` count for cost purposes.

7 Scoring generated code — evaluator.py

7.1 Extracting code from the model’s answer

`extract_code()` first looks for a fenced `'''cpp` or `'''c++` block via regular expression; failing that, any generic triple-backtick block; failing that, it uses the raw response text as-is. This exists as a safety net in case the model ignores the “no markdown” directive.

7.2 Compiling

`_compile()` writes the code to a temporary `.cpp` file and shells out to:

```
g++ -O2 -std=c++23 -o <binary> <source.cpp>
```

with a `COMPILE_TIMEOUT` of 15 seconds. If compilation fails or times out, the individual scores 0 for that problem with reason `compile_fail`, and up to 500 characters of the compiler’s `stderr` are recorded for debugging. On timeout, the whole process group is killed (`os.killpg` with `SIGKILL`) so no orphaned compiler process is left running.

Term: C++23 / g++

C++23 is a specific, recent version of the C++ programming-language standard; `g++` is the GNU C++ compiler, a program that turns C++ source text into a runnable binary. Requiring `-std=c++23` means the generated code must be valid under this particular standard’s rules.

7.3 Running against test cases

`run_tests_verbose()` runs the compiled binary once per test case, feeding it the test’s input on `stdin` (standard input) and capturing what it writes to `stdout` (standard output), each within `EXEC_TIMEOUT` (2 seconds). Output is compared to the expected answer using `_match()`, which normalises both strings (`_normalise`: strip trailing newline, split into lines, and strip trailing whitespace from each line) before an `exact` string comparison.

Flagged for the owner

This exact-match comparison is explicitly discussed in the README as a known, deliberate limitation: a program that is logically correct but prints a floating-point number with different precision, or picks a different but equally valid answer among multiple correct outputs, is scored as a failure. This uniformly deflates every prompt’s score by roughly the same amount, but it also adds noise the GA has to search through. This is documented, not something this explainer inferred.

7.4 Early stopping per problem

If `EARLY_STOP_FAILURES` (3) consecutive test cases fail in a row, the remaining test cases for that problem are skipped and the problem is scored on however many ran (`reason` = `"all_fail_early_stop"` if none passed at all). This saves compute/time on a program that is clearly wrong, at the cost of not running every test.

7.5 Score reasons

Every evaluation is tagged with exactly one `score_reason`, used throughout the results data and the comparison figures:

Reason	Meaning
<code>perfect</code>	every test case passed
<code>partial_pass</code>	some but not all test cases passed
<code>all_fail</code>	every test case ran and failed
<code>all_fail_early_stop</code>	stopped after 3 straight failures, all failed
<code>compile_fail</code>	the generated C++ did not compile
<code>empty_response</code> / <code>empty_code</code>	the model returned nothing usable
<code>infra_failure</code>	all API retries exhausted (network/quota, not the model's fault)

8 Cost accounting — `cost_tracker.py`

Every LLM call anywhere in the project (fitness generation, crossover, all three mutation types) is passed through `tracker.record()`, a thread-safe (`threading.Lock`-protected) singleton that accumulates: total calls, total input/output tokens, and total cost in USD, broken down by **phase** (e.g. "fitness", "crossover") and by **model**. After every single call it rewrites a live JSON summary file (`costs.live.json` inside the run directory, plus a copy in `Costs/`) so a running process can be watched from outside without interrupting it. `restore_from_jsonl()` lets a resumed run rebuild its running totals from the `calls.jsonl` log of a previous, interrupted session.

Flagged for the owner

`cost_tracker.py`'s `PRICING` dictionary still contains entries for Claude models (`claude-haiku-4-5`, `claude-sonnet-4-6`) that this project does not call anywhere in the current code — leftover from an earlier iteration that used a different backend, per the README's own "Challenges" section. It is dead data, not a bug that affects results (the actual per-call cost passed to `tracker.record()` in `llm.py` is computed separately using its own `_pricing` dict for Gemini models only).

9 Entry point and orchestration — `main.py`

`main()` runs, in order: (1) a **pre-flight check** (service-account file exists, required Python packages import, dataset directories exist) that exits with a clear error rather than failing deep inside a multi-hour run; (2) loads and splits the problem pool; (3) either starts a fresh run directory (`results/run-<timestamp>_<scheme>`) or restores one via `--resume`; (4) wires up `SIGINT`/`SIGTERM` signal handlers so Ctrl-C flushes the cost report before exiting; (5) calls `ga.run()`; (6) once the GA returns its best individual, evaluates that single best prompt on the 200-problem holdout set (`evaluate_on_holdout`, using the same `ThreadPoolExecutor` pattern as fitness evaluation) and writes `summary.json`.

Term: Pre-flight check

A validation step performed *before* the main work starts, checking that every external dependency the program needs (files, credentials, libraries) is actually present, so failures are reported immediately and clearly instead of partway through an expensive operation.

10 Analysis scripts

10.1 `run_baseline_eval.py`

Evaluates two fixed prompts — the tournament run’s winning evolved prompt (`results/run_20260429_210411/best_prompt.txt`) and the strongest hand-written seed (`SEED_PROMPTS[0]`) — on the same 200-problem holdout split, writing raw per-problem results to `.jsonl` logs and a summary `comparison.json` with per-rating and per-problem breakdowns. It supports resuming: any problem ID already present in the log file is skipped rather than re-run.

10.2 `visualize_test_results.py`

An entirely offline script (no API calls) that reads the four `*test_evaluations.jsonl` logs (tournament, roulette, elitism evolved prompts, plus baseline) and regenerates five `matplotlib` figures in `results/comparison/`:

1. **Overall score** — one bar per scheme vs. baseline, mean pass rate.
2. **Outcome distribution** — stacked bars of `score_reason` counts.
3. **Score by difficulty rating** — grouped bars per Codeforces rating tier.
4. **Per-problem scatter** — evolved score vs. baseline score, one point per problem, split into three panels (one per scheme).
5. **Win/tie/loss** — how many problems each evolved prompt won, tied, or lost against the baseline.

It asserts each log has exactly 200 entries and that all four logs cover the identical set of problem IDs before plotting, failing loudly rather than silently plotting a partial or mismatched comparison.

11 Results, as actually recorded in this repository

11.1 Headline numbers

Prompt	Evolution-pool fitness (65 probs., best-of-run)	Holdout mean (200 probs.)
Tournament evolved (run_20260429_210411)	0.6531	0.7857
Roulette evolved (run_20260430_014526)	0.6619	0.7478
Elitism evolved (run_20260430_014530)	0.7346	0.7768
Baseline (best hand-written seed, seed 0)	—	0.7900

These are the exact values read from each run’s `summary.json` and from `results/comparison/comparison.json`. `comparison.json` records the tournament-evolved-vs-baseline outcome distribution over the 200 test problems as: evolved {perfect: 142, partial_pass: 44, all_fail_early_stop: 4, compile_fail: 10} vs. baseline {perfect: 142, partial_pass: 44, all_fail_early_stop: 2, compile_fail: 12} — nearly identical outcome mixes, with the evolved prompt compiling 2 fewer times.

Honest headline: evolution did not beat the best hand-written seed

On the 200-problem holdout, every evolved prompt scored *below* the strongest hand-written seed (0.7900). Tournament selection came closest (0.7857) and was the only scheme with a positive per-problem win/loss record against the baseline (26 wins vs. 22 losses out of 200, the rest ties). This is stated plainly in the project’s own README and is not something this explainer is softening or inflating.

11.2 Cost

The three full runs together made roughly 13,000 LLM calls and cost about \$17.05 in Vertex AI charges (\$5.99 + \$6.16 + \$4.90, one report per run under `Costs/`).

11.3 The five comparison figures

`results/comparison/test_1_overall_score.png` through `test_5_win_loss_tie.png` present, respectively: the four prompts’ overall mean scores as a bar chart; a stacked outcome breakdown; grouped bars by difficulty rating; a three-panel scatter of evolved vs. baseline score per problem (points above the diagonal are evolved wins); and a three-panel win/tie/loss count chart. These are regenerated deterministically by `visualize_test_results.py` from the raw `.jsonl` logs — nothing in them is hand-drawn or estimated.

12 Known limitations (from the project’s own documentation)

These are drawn directly from the README’s “Challenges” and “What I’d do differently” sections — restated here, not newly discovered:

- **Population of 5 is very small.** With only 5 individuals across 12 generations, a single unlucky or lucky offspring shifts the population mean noticeably; the scheme-to-scheme differences on the 200-problem test (tournament 0.7857, roulette 0.7717 per the README summary text, elitism 0.7713) sit inside the noise a population this small produces.

- **The 700-token operator cap truncates prompts mid-sentence.** The tournament run’s winning prompt (`best_prompt.txt`) literally ends “... variable types, edge” — cut off by `API_FALLBACK_MAX_TOKENS_0`. A truncated prompt still won its run, meaning the fitness signal tolerated the truncation, but operator output was not validated for completeness before entering the population.
- **No statistical significance testing.** The evolved-vs-baseline gap is a fraction of a percentage point over 200 problems; the comparison script computes means and win/loss counts but no paired significance test (e.g. a bootstrap or paired test over problems).
- **Exact-match scoring is strict.** Floating-point precision differences or multiple valid correct answers are scored as failures (§7).
- **Hardcoded configuration.** Model names, GCP region, and the service-account path live as constants in `config.py` rather than environment variables or CLI flags; likewise `g++` is a hardcoded binary name in `evaluator.py`.

Discrepancy flagged for the owner — not resolved here

The accompanying course paper’s abstract (`Evolving_LLM_Prompts_for_Code_Generation....tex`) states: “*Preliminary results demonstrate that evolved prompts outperform hand-crafted baselines... achieving meaningful improvements in pass@1 scores.*” The results actually committed in this repository (`results/comparison/comparison.json`, and the README’s own honest summary) show the *opposite* on the 200-problem holdout set: every evolved prompt scored below the strongest hand-written baseline. This explainer does not know which numbers the paper’s abstract was written against (an earlier/smaller test set, a different metric, or simply an optimistic framing written before the full 200-problem comparison was run) — it is flagging the mismatch rather than guessing which document is “right,” since the README explicitly instructs against fabricating a resolution. This is worth confirming directly with the paper’s co-author or by checking for an earlier results file, if it matters for how the paper is presented.

13 Glossary

Term	Meaning
Chromosome	The candidate solution a GA evolves; here, an English prompt string.
Population	The full set of candidate solutions alive in one generation.
Generation	One round of scoring the population and breeding a replacement.
Fitness	A single number scoring how good a candidate is (here, mean pass rate).
Selection	Choosing which individuals become parents, biased toward higher fitness.
Crossover	Combining two parents’ traits into one offspring.
Mutation	A small, partly-random change applied to an offspring.
Elitism	Always keeping the best individual(s) unchanged into the next generation.
Pass@1	Standard code-generation metric: does the model’s first (only) attempt pass.
Pass rate / fitness here	Fraction of a problem’s test cases the generated program passes.

Vertex AI Service account	Google Cloud's managed platform for calling hosted AI models. A non-human Google Cloud identity authorized via a private-key file.
Token	A sub-word text chunk; the unit LLM usage and cost are measured in.
Thread pool	A fixed set of worker threads used to run tasks concurrently.
Exponential backoff	Retrying with progressively longer waits between attempts.
Coefficient of variation	Standard deviation divided by mean; a measure of spread.
Stratified split	Splitting a dataset so each subgroup is proportionally represented.
Checkpoint	A saved snapshot of state (population + RNG) enabling resume.
Holdout set	Data reserved and never used for the decisions being tested with it.
